

第一节课 · 网页基础与 AI 辅助开发

手写资料

1 工具下载与安装

工具	Windows	macOS	验证命令
VS Code	https://code.visualstudio.com/ 下载.exe 安装包	https://code.visualstudio.com/ 下载.zip 或 <code>brew install --cask visual-studio-code</code>	<code>code</code> <code>--version</code>
Node.js	https://nodejs.org/ 下载.msi (LTS 版)	https://nodejs.org/ 下载.pkg (LTS 版) 或 <code>brew install node</code>	<code>node -v</code>
Git	https://git-scm.com/downloads 下载.exe	https://git-scm.com/downloads 下载.dmg 或 <code>brew install git</code>	<code>git</code> <code>--version</code>
Claude Code CLI	终端(管理员)运行 <code>npm install -g @anthropic-ai/claude-code</code>	终端运行 <code>npm install -g @anthropic-ai/claude-code</code>	<code>claude</code> <code>--version</code>
ccswitch	https://github.com/farion1231/cc-switch/releases 下载 Win- dows.msi	https://github.com/farion1231/cc-switch/releases 下载 ma- cOS.dmg 或 <code>brew install --cask cc-switch</code>	安装后打开即可

2 ccswitch + DeepSeek 配置

1. 打开 <https://github.com/farion1231/cc-switch/releases> 下载 ccswitch 安装包
2. 安装后打开 ccswitch
3. 点击 « 添加供应商 » → 选择 DeepSeek
4. 去 <https://platform.deepseek.com/> 注册 → API Keys → 创建一个 Key, 复制
5. 把 API Key 粘贴到 ccswitch → 点击 « 启用 »
6. 打开终端, 输入 `claude` 启动, 看到欢迎界面即成功

3 常用终端命令

命令	作用
<code>cd 文件夹名</code>	进入文件夹
<code>cd ..</code>	返回上级文件夹
<code>mkdir 文件夹名</code>	创建文件夹
<code>ls (Mac/Linux) / dir (Windows)</code>	查看当前目录的文件
<code>pwd</code>	查看当前路径
<code>claude</code>	启动 Claude Code
<code>code .</code>	用 VS Code 打开当前文件夹

4 推荐的提示词

4.1 第一条（创建项目）

请帮我创建一个静态网页项目，使用 HTML、CSS 和 JavaScript。
页面主题是：[你的主题]。
页面包含：导航栏、Hero 区、三个功能卡片、案例展示区、
行动按钮和页脚。
风格简洁现代，适配手机和电脑。
请创建 `index.html`、`style.css` 和 `script.js`，
并解释每个文件的作用。

4.2 修改样式

请帮我把页面颜色改得更专业，按钮更明显，卡片层级更清楚。

4.3 优化手机端

请检查这个网页在手机屏幕下是否会拥挤，
并优化响应式布局。

4.4 解释代码

请用适合零基础学生的方式解释这段代码的作用。

5 课堂任务步骤

1. 创建项目文件夹（路径不要有中文）
2. 打开 ccswitch，确保已启用 DeepSeek
3. 启动 Claude Code，输入第一条提示词
4. AI 生成后，找到 `index.html`，双击用浏览器打开
5. 让 AI 继续修改颜色、布局
6. 按 F12 → 切换设备工具栏，检查手机端效果

6 GitHub 从零开始

6.1 第一步：注册 GitHub 账号

打开 <https://github.com/signup> → 输入邮箱、密码、用户名 → 验证邮箱。

记住你的用户名，后面一直要用。

6.2 第二步：告诉 Git 你是谁

安装完 Git 后，第一件事是设置你的名字和邮箱。打开终端运行：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱@example.com"
```

邮箱用注册 GitHub 那个。设好后不会每次再问。

6.3 第三步：生成 SSH 密钥，连接 GitHub

以后推拉代码不想每次输密码，就配 SSH。

生成密钥：

```
ssh-keygen -t ed25519 -C "你的邮箱@example.com"
```

一路按回车（不用设密码）。

查看公钥内容：

Mac/Linux：

```
cat ~/.ssh/id_ed25519.pub
```

Windows：

```
type %USERPROFILE%\\.ssh\id_ed25519.pub
```

把输出的内容全选复制。

添加到 GitHub：

打开 <https://github.com/settings/keys> → New SSH Key → 标题随便填 → 粘贴公钥 → Add SSH Key。

测试连接：

```
ssh -T git@github.com
```

看到 Hi 你的用户名！就成功了。

6.4 第四步：创建一个 GitHub 仓库

打开 <https://github.com/new>

- Repository name 填项目名，比如 my-first-website

- 选 Public（所有人都能看到）
- 不要勾任何选项
- 点击 Create repository

创建后会跳转到一个页面，里面有仓库地址，记住 SSH 格式的那个（以 `git@github.com:` 开头）。

6.5 第五步：把本地项目推送到 GitHub

在终端进入你的项目文件夹，运行：

```
# 把当前文件夹变成 Git 仓库
```

```
git init
```

```
# 把所有文件加入暂存区
```

```
git add .
```

```
# 保存第一个版本
```

```
git commit -m "第一次提交"
```

```
# 关联远程仓库（地址换成你自己的）
```

```
git remote add origin git@github.com:你的用户名/my-first-website.git
```

```
# 推送到 GitHub
```

```
git push -u origin main
```

刷新 GitHub 页面，你的代码已经在那里了。

6.6 第六步：从 GitHub 下载仓库到本地

```
git clone git@github.com:你的用户名/仓库名.git
```

会在当前目录创建一个文件夹，里面就是仓库的所有文件。

6.7 第七步：日常开发四步走

以后每次修改代码都走这四步：

1. `git add .` ——告诉 Git 我要保存这些修改
2. `git commit -m "做了什么修改"` ——打包成一个版本
3. `git pull` ——先拉取 GitHub 上的最新代码（别人可能改了）
4. `git push` ——把我的修改推上去

命令	作用
<code>git status</code>	查看当前有哪些文件被修改过
<code>git log</code>	查看提交历史
<code>git diff</code>	查看具体改了哪些内容

6.8 第八步：分支

分支就是在主线上分出一条岔路，各改各的，互不影响。

查看所有分支（* 表示当前所在分支）

```
git branch
```

创建并切换到新分支

```
git checkout -b 功能名称
```

切换回 main 分支

```
git checkout main
```

把功能分支的修改合并到当前分支

```
git merge 功能名称
```

命令	作用
<code>git branch</code>	查看所有分支
<code>git checkout -b 分支名</code>	创建并切换到新分支
<code>git checkout 分支名</code>	切换到已有分支
<code>git merge 分支名</code>	把指定分支合并到当前分支
<code>git tag 标签名</code>	给当前版本打标签，如 <code>git tag v1.0</code>
<code>git stash</code>	暂时保存当前修改，清空工作区
<code>git stash pop</code>	恢复上一次暂存的修改

6.9 第九步：五个人协作一个仓库

6.9.1 仓库分支结构

main（主分支，始终保持可运行）

dev（开发分支，大家在这里合代码）

feature/你的名字1（个人分支）

feature/你的名字2（个人分支）

feature/你的名字3（个人分支）

feature/你的名字4（个人分支）

规则：永远不在 main 上直接改。每个人在自己的 feature 分支上开发，提 PR 让其他人 review 后合并。

6.9.2 协作流程

第 1 步：克隆仓库

```
git clone git@github.com:你们组/项目名.git
```

第 2 步：创建自己的分支

```
git checkout -b feature/你的名字
```

分支名用英文，比如 feature/alice、feature/bob。

第 3 步：在自己的分支上开发

```
git add .  
git commit -m "完成了导航栏"  
git push origin feature/你的名字
```

第 4 步：提 Pull Request (PR)

在 GitHub 网页上操作：

1. 打开仓库页面，点 Pull Requests → New Pull Request
2. base 选 dev，compare 选 feature/你的名字
3. 写清楚改了什么东西，点 Create Pull Request
4. 找另一个组员 Review 代码，审核通过后点 Merge

第 5 步：同步最新代码

每次开始工作前和合并完别人的 PR 后：

```
git checkout dev  
git pull  
git checkout feature/你的名字  
git merge dev
```

第 6 步：解决冲突

如果 git merge dev 提示冲突：

1. 打开有冲突的文件，找到 <<<<<<< 和 >>>>>>> 标记的位置
2. 和对方商量保留哪段代码
3. 删掉冲突标记和不要的代码
4. 保存文件，然后：

```
git add .
```

```
git commit -m "解决与 dev 的冲突"
```

6.10 第十步：Claude Code 配合 GitHub

在 Git 仓库目录下启动 Claude Code，它会自动识别 Git 上下文。可以直接用自然语言让 AI 帮你操作：

- "看看现在有什么修改" ——相当于 `git status`
- "帮我提交所有修改，说明是'完成了导航栏布局'" ——相当于 `git add . + git commit`
- "推送到远程仓库" ——相当于 `git push`
- "拉取最新代码" ——相当于 `git pull`
- "创建一个新分支叫 `feature/login`" ——相当于 `git checkout -b feature/login`
- "帮我看看最近的提交记录" ——相当于 `git log`